

Python: Data input

D. Leeuw

9 juni 2026

0.0.0

© 2026 Dennis Leeuw



Dit werk is uitgegeven onder de Creative Commons BY-NC-SA Licentie en laat anderen toe het werk te kopiëren, distribueren, vertonen, op te voeren, en om afgeleid materiaal te maken, zolang de auteurs en uitgever worden vermeld als maker van het werk, het werk niet commercieel gebruikt wordt en afgeleide werken onder identieke voorwaarden worden verspreid.

1 Over dit document

1.1 Leerdoelen

Na bestudering van dit document kan de lezer omgaan met:

- Data opgevraagd bij een gebruiker
- Data aangeleverd via argumenten
- Data aangeleverd via opties
- Data aangeleverd via een pipe
- Data aangeleverd via een bestand

1.2 Voorkennis

2 Introductie

Automatisering gaat om data verwerking. We hebben in voorgaande lessen gezien dat Python data kan opslaan in variabelen en daar kunnen we dan bewerkingen op los laten. Ook hebben we data opgevraagd aan de gebruiker via de `input` functie. Er zijn echter nog vele andere manieren waarop we data kunnen binnenhalen binnen ons programma. We kunnen bijvoorbeeld een bestand inlezen of data aangeleverd krijgen via opties. Dit document gaat over deze manieren om data op te nemen in een programma en daarna deze data te verwerken.

3 Data via user input

Het vragen om input van de gebruiker.

```
n = input('Geef een getal op: ')
print(n)
```

De input mag alles zijn: een nummer, een letter, een string of een compleet word-document.

Het opstarten kan met:

```
python user-input.py
```

Om te zorgen dat we iets meer controle hebben over de data die we krijgen kunnen we het beperken tot alleen de getallen:

```
n = int(input('Geef een getal op: '))

print(n)
```

Het opstarten kan met:

```
python user-input-int.py
```

4 Data via argumenten

We hebben bij functies gezien dat deze argumenten kunnen accepteren, dat kan ook bij scripts. Om de argumenten uit te kunnen lezen hebben we een module nodig die **sys** heet. In een list met de naam **argv** zit de bestandsnaam (**argv[0]**) en de argumenten.

Een programma om alle argumenten weer te geven:

```
import sys

if len(sys.argv) < 2:
    print("ERROR: Er zijn geen argumenten opgegeven")
    exit(1)

# argv[0] is de naam van het script
# De argumenten beginnen vanaf index 1
for i in range(0, len(sys.argv)):
    print(sys.argv[i])
```

Het opstarten kan met:

```
python argv.py a b c
```

5 Data via opties

Het nadeel van argumenten is dat het ongestructureerde data is. De input kan van alles zijn. Om iets meer richting aan de data te geven kunnen we gebruik maken van opties.

Een programma om via opties argumenten weer te geven:

```
import argparse

# We maken een object dat opties afhandeld
parser = argparse.ArgumentParser(
    description="Voorbeeld script met opties"
)
```

```
# We voegen opties toe
parser.add_argument("-n", required=True, type=int)
parser.add_argument("-s", required=True, type=str)
parser.add_argument("-o", required=False, type=int)

# Verwerk alle argumenten
args = parser.parse_args()

# Laat zien wat we hebben
print(args)
print(args.n)
print(args.s)
print(args.o)
```

Door gebruik te maken van de `argparse` module krijgen we meteen al 1 gratis feature. Onze code kent meteen de `-h` en de `--help` opties.

```
python opties.py -h
```

De optie geeft de gezette **description** terug en ook welke opties het script ondersteunt. Bij **usage** wordt er duidelijk gemaakt welke opties gezet moeten zijn en welke er mogen zijn.

We kunnen het script nu opstarten met opties

```
python opties.py -n 1 -s hello -o 3
```

We hebben gezien dat we door opties te gebruiken kunnen bepalen wat voor soort data we binnen krijgen en we kunnen bepalen wat meegegeven moet worden en wat niet. Hiermee hebben we al veel meer controle over de data die we kunnen gebruiken binnen een script.

6 Data via bestanden

6.1 Bestanden: File handles

Met Python kan je data van bestanden lezen en data naar bestanden schrijven dit onderdeel gaat over deze mogelijkheden. We gaan eenvoudige data lezen en schrijven van en naar een bestand.

Het werken met bestanden bestaat eruit dat we een bestand eerst moeten openen, daarna moeten we ervan lezen of naar schrijven, waarna we het bestand weer moeten sluiten. Om niet elke keer te hoeven aangeven om welk bestand het gaat gebruiken we een zogenaamde file handle. Je geeft in je script één keer aan welk bestand je wilt openen en koppelt daaraan een file

handle en daarna gebruik je alleen nog deze file handle om te lezen of te schrijven. Tot slot moet je die file handle sluiten.

In Python ziet dat er ongeveer zo uit:

```
# File handle korten we af als fh
fh = open('testbestand.txt')
fh.close()
```

In dit voorbeeld is `fh` de file handle en die heeft functies zoals `write` (schrijven) naar en `close` (sluiten) van een bestand.

Bij het uitvoeren van deze code zal je een error melding krijgen:

```
FileNotFoundError: [Errno 2] No such file or directory: 'testbestand.txt'
```

Deze foutmelding is een runtime error en kan afgevangen worden met een `try` en `except`.

Python neemt standaard aan dat je een tekstbestand wilt openen om het te lezen (`read`). Je kan aan `open()` ook een 2de parameter meegeven, waarmee je zegt wat er gebeuren moet met een bestand:

parameter	functie	werking
r	Read	Opent een bestand om het te lezen, geeft een error als de file niet bestaat. Dit is de default.
w	Write	Opent een bestand om het te schrijven, maakt het bestand aan als het niet bestaat en overschrijft eventueel bestaande inhoud
a	Append	Opent een bestand om er data aan toe te voegen, maakt het bestand aan als het niet bestaat.
x	Create	Maakt een nieuw, leeg, bestand aan, geeft een error als het bestand bestaat.

Tabel 1: Logical operators

Naast deze functies kunnen we in dezelfde 2de parameter ook meegeven of het om een tekst (`t`) of een binair (`b`) bestand gaat. Binaire bestanden zijn bestanden zoals plaatjes of filmpjes. Tekst bestanden zijn bestanden zoals HTML-pagina's of configuratiebestanden. Met de parameter `rb` gaat Python ervanuit dat je een binair bestand wilt gaan lezen. En met `wt` dat je een tekstbestand wilt gaan schrijven.

Tenslotte is er nog een 3de parameter die we meekunnen geven en dat is de encoding, hiermee kunnen we aangeven wat voor character encoding er gebruikt is in een bestand:

```
# File handle korten we af als fh
fh = open('testbestand.txt', 'rt', encoding="utf-8")
fh.close()
```

6.2 Schrijven naar een bestand

We beginnen met het schrijven van een bestand zodat we iets hebben om mee te werken als we een bestand willen gaan lezen:

```
try:
    fh = open("testbestand.txt", "wt")
except fh.FileNotFoundError:
    print("Bestand niet gevonden")

fh.write("Dit is de eerste regel van ons nieuwe bestand.")
fh.write("Dan is dit dus de tweede regel van ons nieuwe bestand.")
fh.write("En tot slotte de derde en laatste van ons nieuwe bestand.")
fh.close()
```

Runnen we de code en bekijken we daarna de inhoud van ons nieuwe bestand dan zien we dat het 1 lange regel geworden is en helemaal geen 3 verschillende regels.

Python voegt kennelijk letterlijk de regels toe aan ons bestand zonder rekening te houden met dat wij willen werken met regels. Op de één of andere manier zullen we Python dus moeten gaan vertellen dat er een regel einde is. Op Unix (Linux, Mac OS X) systemen is het voldoende om aan te geven dat er een new-line is, op Windows systemen geldt de regel dat er een return + newline moet zijn. Deze extra elementen kunnen we toevoegen in onze code voor een new-line gebruiken we `\n` en voor een return gebruiken we `\r`.

```
try:
    fh = open("testbestand.txt", "wt")
except fh.FileNotFoundError:
    print("Bestand niet gevonden")
    exit()

fh.write("Dit is de eerste regel van ons nieuwe bestand.\r\n")
fh.write("Dan is dit dus de tweede regel van ons nieuwe bestand.\r\n")
fh.write("En tot slotte de derde en laatste van ons nieuwe bestand.\r\n")
fh.close()
```

We hebben in de voorgaande code gezien dat na elke open er ook een close **moet** zijn. Het risico bestaat dat als er ergens een fout optreedt en het script crashed dat het dan nooit bij de close gekomen is. Hierbij kan er

data verloren gaan. Om dit te voorkomen is er een speciale constructie en dat is de `with`:

```
with open('testbestand2.txt', 'wt', encoding="utf-8") as fh:
    fh.write("We springen in bij het gebruik van with.\r\n")
    fh.write("Zodra het inspringen over is zal het bestand gesloten
    worden.\r\n")
# Rest van de code
```

Het advies is dan ook om altijd de constructie `with open(...) as fh:` te gebruiken.

6.3 Lezen van een bestand

Nu we een bestand geschreven hebben, gaan we kijken hoe we dit bestand weer kunnen inlezen.

```
file='testbestand.txt'

try:
    with open(file, 'rt', encoding="utf-8") as fh:
        read_data = fh.read()
except FileNotFoundError:
    print(f"Bestand {file} niet gevonden")
    exit()
except PermissionError:
    print(f"Geen rechten on {file} te lezen")
    exit()

print(type(read_data))
print(read_data, end='')
```

De variabele `data_read` bevat het volledige document. Dit kan een probleem veroorzaken als het bestand groter is dan de maximaal beschikbare vrije geheugenruimte! We gaan later zien hoe we dit kunnen voorkomen.

Het boven getoonde script toont een paar nieuwe zaken die we nog niet kennen van Python, dus we lopen het script eerst even door. Het script met het maken van een variabele die de bestandsnaam bevat. Door dit in een variabele te doen kunnen we het script later makkelijker wijzigen als we een ander bestand willen lezen.

Via “try/except” openen we een bestand en met `read` lezen we alle data uit het bestand in het geheugen. Als dit allemaal goed verlopen is dan verlaten we de “try/except” constructie. Omdat `with` hebben gebruikt hoeven het bestand niet te sluiten, daar zorgt Python voor. Omdat Python het bestand voor ons sluit bestaat de file handle niet meer als we bij de `except` aankomen. De `except` checked dan ook direct op de error en niet op de `fh.error`.

Tot slot printen we eerst wat `read_data` eigenlijk is voor variabele en het blijkt een string te zijn. Daarna printen we de variabele met `print`. We gebruiken daarbij een speciale constructie. We vertellen print namelijk dat deze moet stoppen als er een lege regel voorbij komt. Dit is een echte lege regel, dus hij mag ook geen new line of return bevatten. Het is de zogenaamde NULL regel. Verwijder maar eens de `end=""` constructie en zie dat er dan een extra lege regel geprint wordt bij het printen van het bestand.

6.4 Lezen van regels uit een bestand

Als eerste is er `readline` om een bestand regel voor regel te lezen. Het leest één regel van een bestand. Een tweede `readline` opdracht leest de volgende regel:

```
file='testbestand.txt'

with open(file, 'rt', encoding="utf-8") as fh:
    print(fh.readline())
    print(fh.readline())
    print(fh.readline())
    print(fh.readline())
    print(fh.readline())
```

Geven we meer `readline` opdrachten dan er regels in een bestand zitten dan zal python geen error geven, maar extra lege regels geven.

We kunnen ook met een `for` loop data uit een bestand lezen:

```
file='testbestand.txt'

with open(file, 'rt', encoding="utf-8") as fh:
    for line in fh:
        print(f"{line}\n", end='')
```

Als we met de data willen werken is het misschien handiger als elke regel uit een bestand als een item in een **list** terecht komt. Zo kunnen we per regel met het bestand werken:

```
file='testbestand.txt'

with open(file, 'rt', encoding="utf-8") as fh:
    file = fh.readlines()

for line in file:
    print(line)
```


6.5 Bestanden verwijderen

Om een bestand weg te gooien (delete) hebben we de `os`-module nodig waaruit we de `remove` functie gebruiken:

```
import os
os.remove("demofile.txt")
```

6.6 Bestanden: Opdrachten

6.6.1 Maken van een notitieblok

Maak een programma dat een bestand (`notes.txt`) kan openen, en een nieuwe regel toevoegt met datum + tekst.

1. Open een bestand `notes.txt` (maak het aan als het niet bestaat).
2. Vraag de gebruiker om een notitie.
3. Voeg de notitie + datum/tijd toe aan het bestand. Met als format eerst de datum dan een komma, dan de tijd en een komma en dan de notitie.
4. Bonus: toon na afloop alle notities in de console.

6.7 Bestanden en directories

Om te zien of een pad of een bestand bestaat op een systeem kunnen we gebruik maken van de `os`-module of van de `pathlib`-module. We zullen van beide een voorbeeld geven. Het heeft de voorkeur om de moderne versie met `pathlib` te gebruiken.

De versie met de `os`-module:

```
import os

pad = "C:/Users/Public/Documents/bestand.txt"

if os.path.exists(pad):
    print("Pad bestaat")
    if os.path.isfile(pad):
        print("Het is een bestand")
    elif os.path.isdir(pad):
        print("Het is een map")
else:
    print("Pad bestaat niet")
```

De versie met de `pathlib`-module:

```
import pathlib

pad = Path("C:/Users/Public/Documents/bestand.txt")

if pathlib.path.exists():
    print("Pad bestaat")
    if pathlib.path.is_file():
        print("Het is een bestand")
    elif pathlib.path.is_dir():
        print("Het is een map")
else:
    print("Pad bestaat niet")
```

We zullen in de rest van dit document alleen nog `pathlib` gebruiken.

6.7.1 Analyseer de inhoud van een map

Laat het programma een map met bestanden inlezen en een overzicht maken in `report.txt`.

1. Vraag aan de gebruiker om een map
2. Test of de map bestaat
3. Haal informatie op van alle bestanden in de map
4. Schrijf in `report.txt`: naam van de map, bestandsnaam, grootte, datum
5. Bonus: Maak een notitie in `notes.txt` met de naam van de map die geanalyseerd is, met natuurlijk de datum en tijd.

6.8 Werken met CSV bestanden

De `csv`-module in Python wordt gebruikt om CSV-bestanden (Comma Separated Values) te lezen en te schrijven. Een CSV-bestand is een tekstbestand waarin gegevens in tabellen worden opgeslagen — elke regel is een rij, en kolommen zijn gescheiden door een komma (of ander scheidingsteken zoals ; of `\t`).

Voorbeeld van een CSV-bestand:

```
naam,leeftijd,stad
Anna,25,Amsterdam
Bram,30,Rotterdam
```

Neem dit bestand over en sla het op als `personen.csv`.

6.8.1 Lezen van CSV-bestanden

Er zijn twee verschillende functies die we uit de CSV-module kunnen gebruiken om een CSV-bestand te lezen. De eerste is `csv.reader()` en de tweede is `csv.DictReader()`. Beide functies zullen we bekijken.

Met `csv.reader()` lees je een bestand regel voor regel en kan je regel voor regel gebruiken:

```
import csv

with open('personen.csv', newline='', encoding='utf-8') as csvfile:
    reader = csv.reader(csvfile)
    for line in reader:
        print(line)
```

Met `csv.DictReader()` lees je een bestand als een dictionary in. Het verwacht dat de eerste regel van het CSV-bestand de namen van de kolommen bevat en geen echte data. Die namen worden gebruikt om per regel een dictionary te maken.

```
import csv

with open('personen.csv', newline='', encoding='utf-8') as csvfile:
    reader = csv.DictReader(csvfile)
    for line in reader:
        print(line['naam'], line['stad'])
```

6.8.2 Functie parameters

Aan de functies kunnen we parameters meegeven. De belangrijkste parameters zijn:

- `delimiter` – scheidingsteken (standaard ,)
- `quotechar` – teken om tekst te omgeven (standaard ")
- `quoting` – manier waarop waarden tussen aanhalingstekens worden gezet
- `newline` – teken dat einde van de regel bepaalt (standaard newline)

6.8.3 Schrijven van CSV-bestanden

Ook voor het schrijven van een CSV-bestand hebben we 2 functies.

Met `csv.writer()` schrijven we platte tekst weg als een CSV-bestand.

```
import csv

with open('personen.csv', 'w', newline='', encoding='utf-8') as csvfile:
    writer = csv.writer(csvfile)
    writer.writerow(['naam', 'leeftijd', 'stad'])
    writer.writerow(['Anna', 25, 'Amsterdam'])
    writer.writerow(['Bram', 30, 'Rotterdam'])
```

Met `csv.DictWriter()` kunnen we een dictionary wegschrijven als CSV-bestand. Dit moet regel voor regel gebeuren en we moeten een header schrijven die de kolomnamen bevat.

```
import csv

with open('personen.csv', 'w', newline='', encoding='utf-8') as csvfile:
    kolomnamen = ['naam', 'leeftijd', 'stad']
    writer = csv.DictWriter(csvfile, fieldnames=kolomnamen)

    writer.writeheader()
    writer.writerow({'naam': 'Anna', 'leeftijd': 25, 'stad': 'Amsterdam'})
    writer.writerow({'naam': 'Bram', 'leeftijd': 30, 'stad': 'Rotterdam'})
```

6.9 CSV bestanden: Opdracht

We hebben tot nu toe elke keer een regel weggeschreven in `rapport.txt` of `notities.txt` die bestond uit de verzamelde data gescheiden door een komma. Een bestand met “Comma Separated Values” heet een CSV-bestand. Voor CSV bestanden gelden meer regels dan alleen dat de waarden gescheiden moeten zijn door een komma. Om te voldoen aan al die regels is er een Python module met de naam `csv` die ervoor zorgt dat de weggeschreven data echt een CSV bestand vormt.

1. Installeer de CSV-module
2. Maak een kopie van je notities applicatie
3. Herschrijf de kopie van de applicatie aan zodat de output een echt CSV bestand is en noem dit output bestand `notities.csv`.
4. Maak een kopie van je rapportage applicatie
5. Herschrijf de kopie van de applicatie zodat de output een echt CSV bestand is en noem dit output bestand `rapport.csv`.

7 Pipe

Data kan ook aangeleverd worden als output van en ander commando. De data kan dan via het pipe-character (|) doorgestuurd worden naar ons script.

Een Python script dat data van een pipe aanneemt en regel voor regel weergeeft op het scherm zou er zo uit kunnen zien:

```
import sys

for line in sys.stdin:
    sys.stdout.write(line)
```

Zoals je ziet hebben we voor de script de sys-module nodig.

Een commando om dit script te testen op een Linux systeem zou er zo uit kunnen zien:

```
echo -e "Mijn eerste regel\nEn een tweede regel" | python pipe.py
```

Op een Windows systeem met PowerShell wordt het:

```
Write-Out ("Mijn eerste regel","En een tweede regel") | python.exe
.\pipe.py
```

7.1 JSON

JSON is een manier om data betekenis te geven. Het is ontstaan op het web. Als we een webpagina hebben met informatie zoals:

```
<p>Jason Argonaut
Jupiterlaan 1
Heelverweg</p>
```

en we vragen die op, dan hebben we wel adres gegevens, maar we kunnen er niet zo veel mee.

De eerste oplossing was om daar XML van te maken:

```
<voornaam>Jason</voornaam> <achternaam>Argonaut</achternaam>
<adres>Jupiterlaan 1</adres>
<plaats>Heelverweg</plaats>
```

Het zorgt voor meer betekenis van de ontvangen data, maar het wordt er niet leesbaarder op.

JSON heeft geprobeerd dit probleem op te lossen:

```
{
  "Voornaam": "Jason",
  "Achternaam": "Argonaut",
  "Adres": "Jupiterlaan 1",
```

```
"Plaats": "Heelverweg"
}
```

staat voor JavaScript Object Notation

Is voor gegevensuitwisseling Menselijk leesbaar Gemakkelijk te parsen
door machines Veel gebruikt in webapplicaties en API's

7.1.1 Een JSON filter

Data kan ook aangeleverd worden als output van een ander commando. De data kan dan via het pipe-character (|) doorgestuurd worden naar ons script.

Een Python script dat data van een pipe aanneemt en regel voor regel weergeeft op het scherm zou er zo uit kunnen zien:

```
import sys
import json

for line in sys.stdin:

    json_data=json.loads(line)

    print(json_data["Naam"])
```

Zoals je ziet hebben we voor de script de sys-module nodig.

Een commando om dit script te testen op een Linux systeem zou er zo uit kunnen zien:

```
echo "echo '{"Naam":"Jason","Leeftijd":42}'" | python pipe-json.py
```

We kunnen de input combineren met het werken met opties, zodat je in een optie de key kan meegeven die je wilt zien:

```
import sys
import json
import argparse

parser = argparse.ArgumentParser(
    description="Filter 1 element uit JSON input"
)

parser.add_argument("--key", required=True, type=str)

args = parser.parse_args()

for line in sys.stdin:

    json_data=json.loads(line)

    print(json_data[args.key])
```